241801053-J B DHEJASVI --ASSIGNMENT 2

1. Maximize $z = -x1 + 3x2$ Subject to $x1 + 2x2 \geq 2$ $3x1 + x2 \leq 3$ $x1 \leq 4$ $x1, x2 \geq 0$ Ans: $x1 = 0$, $x2 = 3$, $z = 9$

```
import numpy as np
import pandas as pd

def simplex_max(c, A, b):
    m, n = A.shape

    tableau = np.zeros((m+1, n+m+1))
    tableau[:m, :n] = A
    tableau[:m, n:n+m] = np.eye(m)
    tableau[:m, -1] = b
    tableau[-1, :n] = -c

    cols = [f"x{i+1}" for i in range(n)] + [f"s{i+1}" for i in range(m)] + ["RHS"]
    df = pd.DataFrame(tableau, columns=cols)

    print("\nInitial Tableau\n", df)


    while True:
        last_row = df.iloc[-1, :-1]
        if (last_row >= 0).all():
            break   # optimal reached

        pivot_col = last_row.idxmin()

        ratios = df.loc[:m-1, "RHS"] / df.loc[:m-1, pivot_col]
        ratios[df.loc[:m-1, pivot_col] <= 0] = np.inf
        pivot_row = ratios.idxmin()

        pivot = df.loc[pivot_row, pivot_col]
        df.loc[pivot_row] /= pivot

        for r in df.index:
            if r != pivot_row:
                df.loc[r] -= df.loc[r, pivot_col] * df.loc[pivot_row]

        print("\nUpdated Tableau\n", df)


    solution = {f"x{i+1}": 0 for i in range(n)}

    for i in range(n):
        col = df.iloc[:m, i]
        if np.isclose(col.sum(), 1) and (col >= 0).all():
            row = col.idxmax()
            solution[f"x{i+1}"] = df.loc[row, "RHS"]

    Z = df.loc[m, "RHS"]
    return solution, Z
```

```
c = np.array([10, 15, 20])
A = np.array([[2, 4, 6],
              [3, 9, 6]])
b = np.array([24, 30])

sol, Z = simplex_max(c, A, b)

print("\nFinal Answer:")
print("x1 =", sol["x1"])
print("x2 =", sol["x2"])
print("x3 =", sol["x3"])
print("Z  =", Z)
```

```
c2 = np.array([-1, 3, -3])
A2 = np.array([[3, -1, 1],
               [-1, 2, 0],
               [-4, 3, 8]])
b2 = np.array([7, 6, 10])

sol2, z2 = simplex_max(c2, A2, b2)

print("Problem 2:")
print("x1 =", sol2["x1"])
print("x2 =", sol2["x2"])
print("x3 =", sol2["x3"])
print("Z  =", z2)
```

```
c3 = -np.array([1, -2, 1])
A3 = np.array([[1, 2, -2],
               [1, 0, -1],
               [2, -1, 2]])
b3 = np.array([4, 3, 2])

sol3, z3 = simplex_max(c3, A3, b3)

print("\nProblem 3:")
print("x1 =", sol3["x1"])
print("x2 =", sol3["x2"])
print("x3 =", sol3["x3"])
print("Z  =", -z3)
```

```
Initial Tableau
      x1    x2    x3   s1   s2   RHS
0   2.0   4.0   6.0  1.0  0.0  24.0
1   3.0   9.0   6.0  0.0  1.0  30.0
2 -10.0 -15.0 -20.0  0.0  0.0   0.0

Updated Tableau
          x1        x2   x3        s1   s2   RHS
0  0.333333  0.666667  1.0  0.166667  0.0   4.0
1  1.000000  5.000000  0.0 -1.000000  1.0   6.0
2 -3.333333 -1.666667  0.0  3.333333  0.0  80.0
```

```
Updated Tableau
     x1    x2   x3            s1        s2    RHS
0  0.0  -1.0  1.0  5.000000e-01 -0.333333    2.0
1  1.0   5.0  0.0 -1.000000e+00  1.000000    6.0
2  0.0  15.0  0.0 -8.881784e-16  3.333333  100.0

Updated Tableau
     x1    x2            x3   s1        s2    RHS
0  0.0  -2.0  2.000000e+00  1.0 -0.666667    4.0
1  1.0   3.0  2.000000e+00  0.0  0.333333   10.0
2  0.0  15.0  1.776357e-15  0.0  3.333333  100.0

Final Answer:
x1 = 10.0
x2 = 0
x3 = 0
Z  = 100.0

Initial Tableau
     x1   x2   x3   s1   s2   s3   RHS
0  3.0 -1.0  1.0  1.0  0.0  0.0   7.0
1 -1.0  2.0  0.0  0.0  1.0  0.0   6.0
2 -4.0  3.0  8.0  0.0  0.0  1.0  10.0
3  1.0 -3.0  3.0  0.0  0.0  0.0   0.0

Updated Tableau
     x1   x2   x3   s1   s2   s3   RHS
0  2.5  0.0  1.0  1.0  0.5  0.0  10.0
1 -0.5  1.0  0.0  0.0  0.5  0.0   3.0
2 -2.5  0.0  8.0  0.0 -1.5  1.0   1.0
3 -0.5  0.0  3.0  0.0  1.5  0.0   9.0

Updated Tableau
     x1   x2   x3   s1   s2   s3   RHS
0  1.0  0.0  0.4  0.4  0.2  0.0   4.0
1  0.0  1.0  0.2  0.2  0.6  0.0   5.0
2  0.0  0.0  9.0  1.0 -1.0  1.0  11.0
3  0.0  0.0  3.2  0.2  1.6  0.0  11.0
Problem 2:
x1 = 4.0
x2 = 5.0
x3 = 0
Z  = 11.0
```

. Minimize $z$ = 2$x$1 + 3$x$2 Subject to $x$1 + $x$2 ≥ 6 7$x$1 + $x$2 ≥ 14 $x$1, $x$2 ≥ 0 Ans: $x$1 = 6, $x$2 = 0, $x$3 = 0, $z$ = 12

```
from scipy.optimize import linprog


c = [2, 3]
A = [
    [-1, -1],
    [-7, -1]
]

b = [-6, -14]

bounds = [(0, None), (0, None)]

result = linprog(c, A_ub=A, b_ub=b, bounds=bounds, method='simplex')
```

```
print("Status:", result.message)
print("x1 =", round(result.x[0], 2))
print("x2 =", round(result.x[1], 2))
print("Minimum z =", round(result.fun, 2))
```

```
Status: Optimization terminated successfully.
x1 = 6.0
x2 = 0.0
Minimum z = 12.0
/tmp/ipython-input-1214449848.py:14: DeprecationWarning: `method='simplex'` is deprecated and will
  result = linprog(c, A_ub=A, b_ub=b, bounds=bounds, method='simplex')
```

3. Minimize $z$ = 4$x$1 + $x$2 Subject to 3$x$1 + $x$2 = 3 4$x$1 + 3$x$2 ≥ 6 $x$1, $x$2, $x$3 ≥ 0 $x$1 + 2$x$2 ≤ 4 Ans: $x$1 = 2/ 5 , $x$2 = 9/ 5 , $z$ = 17/ 5

```
from scipy.optimize import linprog

c = [4, 1]

A_ub = [
    [-4, -3],   # 4x1 + 3x2 ≥ 6 → -4x1 -3x2 ≤ -6
    [1, 2]      # x1 + 2x2 ≤ 4
]

b_ub = [-6, 4]

A_eq = [
    [3, 1]      # 3x1 + x2 = 3
]

b_eq = [3]


bounds = [(0, None), (0, None)]

result = linprog(c, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method='simplex')


print("Status:", result.message)
print("x1 =", round(result.x[0], 4))
print("x2 =", round(result.x[1], 4))
print("Minimum z =", round(result.fun, 4))
```

```
Status: Optimization terminated successfully.
x1 = 0.4
x2 = 1.8
Minimum z = 3.4
/tmp/ipython-input-3265848454.py:21: DeprecationWarning: `method='simplex'` is deprecated and will
  result = linprog(c, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method='simplex')
```